ACTIVITY-BASED LEARNING FRAMEWORK

Python Programming Laboratory Manual

Course Code & Execution Workbook for Practical Mastery

Course Title	Activity-Based Learning: Python Programming
Document Code	ABL-PY-2026-MANUAL
Target System	Python 3.x / IDLE / Jupyter
Standard Format	Aim | Procedure | Algorithm | Code | Output | Conclusion

EXPERIMENT 01: BASIC INPUT/OUTPUT & ARITHMETIC OPERATIONS

ABL-01

1. AIM

To write and execute a Python program that reads two numeric values from the user and computes their sum, difference, product, division, and modulo remainder.

2. PROCEDURE

1. Open Python IDLE or a code editor and create a new file named `arithmetic_ops.py`.
2. Prompt the user to input two numbers using `input()` and parse them via `float()`.
3. Perform addition, subtraction, and multiplication operations.
4. Check if the second number is non-zero before division and modulo to prevent runtime errors.
5. Format and output results using f-strings.

3. ALGORITHM

1. Start program execution.
2. Input variables num1 and num2 from user.
3. Compute sum, difference, and product.
4. If num2 != 0, compute division and modulo; otherwise display warning.
5. Print calculated results and stop.

6. PROGRAM CODE

```
      # Experiment 01: Arithmetic Operations in Python num1 = float(input("Enter first number: ")) num2 = float(input("Enter second number: ")) print(f"Addition: {num1} + {num2} = {num1 + num2}") print(f"Subtraction: {num1} - {num2} = {num1 - num2}") print(f"Multiplication: {num1} * {num2} = {num1 * num2}") if num2 != 0: print(f"Division: {num1} / {num2} = {num1 / num2:.2f}") print(f"Modulus: {num1} % {num2} = {num1 % num2}") else: print("Error: Division by zero is undefined.")
```

EXPERIMENT 01 (CONTINUATION)

ABL-01

7. OUTPUT

```
Enter first number: 24 Enter second number: 5 Addition: 24.0 + 5.0 = 29.0
Subtraction: 24.0 - 5.0 = 19.0 Multiplication: 24.0 * 5.0 = 120.0 Division: 24.0 / 5.0
= 4.80 Modulus: 24.0 % 5.0 = 4.0
```

8. CONCLUSION

The program successfully performs arithmetic operations based on user input while properly taking into account divide-by-zero validation logic.

EXPERIMENT 02: PRIME NUMBER DETERMINATION

ABL-02

1. AIM

To design a Python program to determine whether a user-given positive integer is a Prime number.

2. PROCEDURE

1. Prompt the user for a positive integer value.
2. Check if the number is less than or equal to 1; if so, identify it as non-prime.
3. Construct a loop starting from 2 up to the square root of the target integer.
4. Test divisibility using modulo operation. Display the final classification.

6. PROGRAM CODE

```
import math num = int(input("Enter a positive integer: ")) is_prime = True if num
<= 1: is_prime = False else: for i in range(2, int(math.isqrt(num)) + 1): if num % i ==
0: is_prime = False break if is_prime: print(f"{num} is a Prime Number.") else:
print(f"{num} is NOT a Prime Number.")
```

7. OUTPUT

```
Enter a positive integer: 29 29 is a Prime Number.
```

8. CONCLUSION

Prime verification logic was successfully implemented using optimized O(sqrt(N)) loop techniques.

EXPERIMENT 03: FIBONACCI SEQUENCE GENERATION

ABL-03

1. AIM

To construct a Python program to generate and print the Fibonacci sequence up to n terms using loop operations.

2. PROCEDURE

1. Read the number of required terms n from the user.
2. Initialize first two terms a = 0 and b = 1.
3. Execute a loop running n times, computing subsequent terms iteratively.

6. PROGRAM CODE

```
```

7. OUTPUT

```
Enter number of terms: 7 Fibonacci sequence: 0 1 1 2 3 5 8
```

8. CONCLUSION

Fibonacci series creation was successfully verified using non-recursive iterative loops.

EXPERIMENT 04: LIST OPERATIONS & SEARCHING

ABL-04

1. AIM

To implement standard list manipulations in Python including sum, average, searching element target, and sorting.

2. PROCEDURE

1. Construct a Python list from space-separated input elements.
2. Compute sum and average. Perform linear search for a target key and sort the list.

6. PROGRAM CODE

```
```

7. OUTPUT

```
Enter integers separated by space: 45 12 89 23 7 Total Sum: 176 Average: 35.20
Enter element to search: 89 Element 89 found at index 2. Sorted List: [7, 12, 23, 45,
89]
```

8. CONCLUSION

Python dynamic list methods were successfully demonstrated for mathematical calculations, key search, and sorting.

EXPERIMENT 05: DICTIONARY CHARACTER FREQUENCY COUNTER

ABL-05

1. AIM

To design a Python script using dictionaries to count the frequency of each unique character in a text string.

2. PROCEDURE

1. Read a text string input from user. Instantiate empty dictionary `freq_dict`.
2. Iterate over each character and update counts using `get()` method.

6. PROGRAM CODE

```
input_string = input("Enter a string: ") freq_dict = {} for char in input_string:
```

7. OUTPUT

```
Enter a string: abl python Character Frequency Map: 'a': 1 'b': 1 'l': 1 ' ': 1
'p': 1 'y': 1 't': 1 'h': 1 'o': 1 'n': 1
```

8. CONCLUSION

Efficient character hashing and key-value counting were successfully demonstrated using Python dictionaries.

EXPERIMENT 06: FILE I/O OPERATIONS & METRIC ANALYSIS

ABL-06

1. AIM

To program Python File I/O operations by creating a text file, writing sample text, reading back contents, and calculating metrics.

2. PROCEDURE

1. Open file in write mode using context manager, write sample lines, re-open in read mode, and analyze metrics.

6. PROGRAM CODE

```
filename = "sample_lab.txt" with open(filename, "w") as file: file.write("Welcome
```

7. OUTPUT

```
File Analysis Results for 'sample_lab.txt': Total Lines: 3 Total Words: 15 Total
Characters: 91
```

8. CONCLUSION

File write, read, and content analysis operations were executed accurately using Python standard file contexts.